



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Centre de la Imatge i la Tecnologia Multimèdia

Lab Session 3: Data serialization



What is data serialization?

What: Converting the data stored in memory into a linear sequence of bytes that can be transmitted over the network layer protocol (e.g. TCP/UDP).

Why: Its purpose is to ensure that both the sending and the receiving ends understand and interpret the data sent, in the same way.

Example

- Problem 1: How to read from receiver?
- What if you are sending a pointer?

```
void send(socket sock, myFavouriteClass obj)
{
    sock.send(Encoding.ASCII.GetBytes(obj.ToString()), recv,
    SocketFlags.None)
}
```

```
class myFavouriteClass
{
    ...
    private int param1;
    private float param2;
}
```

Streams

- Data structure that encapsulates an ordered set of elements that programs can read or write in sequence.
- Memory Streams
 - Encapsulate a buffer in memory, and give write/read access
 - (in other languages you can have separate `InputMemoryStream` and `OutputMemoryStream` classes with exclusive read or write access)

MemoryStream: <https://docs.microsoft.com/en-us/dotnet/api/system.io.memorystream?view=net-5.0>

Serialize

```
void serialize()
{
    double myfloat=100.1f;
    int myint=15;
    string mystring = "test";
    int[] mylist = new int[3]{1,2,4};
    stream = new MemoryStream();
    BinaryWriter writer = new BinaryWriter(stream);
    writer.Write(myfloat);
    writer.Write(myint);
    writer.Write(mystring);
    foreach (var i in mylist)
    {
        writer.Write(i);
    }
    Debug.Log("serialized!");
}
```

Deserialize

```
void deserialize()
{
    BinaryReader reader = new BinaryReader(stream);
    stream.Seek(0, SeekOrigin.Begin);
    double newfloat=reader.ReadDouble();
    Debug.Log("float "+newfloat.ToString());
    int newint = reader.ReadInt32();
    Debug.Log("int "+newint.ToString());
    string newstring = reader.ReadString();
    Debug.Log("string "+newstring.ToString());
    int[] newlist=new int[3];
    for (int i=0;i<newlist.Length;i++)
    {
        newlist[i]=reader.ReadInt32();
    }
    Debug.Log(newlist.ToString());
}
```

Endianness

- Order in which the bytes of a number are encoded:
- Big-endian: larger (most significant) bytes first
- Little-endian: smaller bytes (less significant) first
- Specially important when communicating across programs made by different people or libraries.
- Usually, 1st time is irrelevant, but affects later developments on the technology

Alternative 1: JsonUtility

```
class testClass
{
    public int hp=12;
    public List<int> pos = new List<int> {3,3,3};
}
```

```
void serializeJson()
{
    var t = new testClass();
    t.hp=40;
    t.pos=new List<int>{10,3,12};
    string json = JsonUtility.ToJson(t);
    stream=new MemoryStream();
    BinaryWriter writer = new BinaryWriter(stream);
    writer.Write(json);
}
```

```
void deserializeJson()
{
    var t = new testClass();
    BinaryReader reader = new BinaryReader(stream);
    stream.Seek(0,SeekOrigin.Begin);
    string json = reader.ReadString();
    Debug.Log(json);
    t = JsonUtility.FromJson<testClass>(json);
    Debug.Log(t.hp.ToString() + " " + t.pos.ToString());
}
```

Alternative 2: XmlSerializer

```
void deserializeXML()
{
    XmlSerializer serializer = new
    XmlSerializer(typeof(testClass));
    var t = new testClass();
    stream = new MemoryStream();
    stream.Write(bytes, 0, bytes.Length);
    stream.Seek(0, SeekOrigin.Begin);
    t = (testClass) serializer.Deserialize(stream);
    Debug.Log("Xml " + t.hp.ToString() + " " +
    t.pos.ToString());
}
```

```
void serializeXML()
{
    var t = new testClass();
    t.hp=40;
    t.pos=new List<int>{10,3,12};
    XmlSerializer serializer = new
    XmlSerializer(typeof(testClass));
    stream=new MemoryStream();
    serializer.Serialize(stream, t);
    bytes=stream.ToArray();
}
```

Alternative 3: BinaryFormatter (Unsafe!)

Serialize:

```
MemoryStream stream = new MemoryStream();
BinaryFormatter formatter = new BinaryFormatter();
try
{
    formatter.Serialize(stream, objectToSerialize);
}
catch (SerializationException e)
{
    Debug.Log("Serialization Failed : " + e.Message);
}
byte[] objectAsBytes = stream.ToArray();
stream.Close();
```

Deserialize:

```
SomeObject objectThatWasDeserialized;
MemoryStream stream = new MemoryStream();
stream.Write(objectAsBytes, 0, objectAsBytes.Length);
stream.Seek(0, SeekOrigin.Begin);
BinaryFormatter formatter = new BinaryFormatter();
try
{
    objectThatWasDeserialized =
        (SomeObject)formatter.Deserialize(stream);
}
catch (SerializationException e)
{
    Debug.Log("Deserialization Failed : " + e.Message);
}
stream.Close();
```

BinaryFormatter:

<https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.formatters.binary.binaryformatter?view=net-5.0>

<https://docs.microsoft.com/en-us/dotnet/standard/serialization/binaryformatter-security-guide>

Warning!

- Don't try to serialize directly GameObjects and classes derived from MonoBehaviour
- Best case:
 - Error when serializing
- Worst case:
 - No error, but the data is not consistent with your scene/project, and “polterbugs” will appear
- Recommendation: Always make your own class to send over the network
- (OR, for the brave ones, make an ISerializable interface)

Other material

- Serialization in Unity:
- <https://docs.unity3d.com/Manual/script-Serialization.html>
- Sample code for serialization

Serialization Challenge

1. Your TCP/UDP delivery is your starting point
2. Create a struct with the following declaration:

```
struct DTO
{
    string playerName;
    int level;
    List<string> ownedPokemons;
}
```

3. Fill it with dummy data (The list should contain at least 2 elements)
4. Create a function Serialize that would transform it to byte[]
5. Create a function Deserialize that would transform it from byte[]
6. Send it from the client to the server
7. Check that the server gets it properly.

Serialization Challenge (2)

1. Create a class with the following declaration

```
class Pokemon
{
    public string name;
}
```

2. Update the previous DTO so that list is now made of Pokemon

```
struct DTO
{
    string playerName;
    int level;
    List<Pokemon> ownedPokemons;
}
```

3. Fill it with dummy data (The list should contain at least 2 elements)
4. Update Serialize/Deserialize functions
5. Send it from the client to the server
6. Check that the server gets it properly.